

Título a ser colocado.

Daniel G. D. Silva¹, Ederson N. F. G. Júnior²

¹ Instituto Federal de Minas Gerais (IFMG) – Campus Ouro Branco
Caixa Postal 36.494-018 – Ouro Branco – Minas Gerais – Brasil

²Bacharelado em Sistema de informação - IFMG Ouro Branco

daniel24maio@gmail.com, ederson.junior@ifmg.edu.br

Abstract. *Colocar Resumo em inglês aqui*

Resumo. *Colocar Resumo aqui*

1. Introdução

A formação em cursos técnicos e de graduação envolve não apenas a aquisição de conhecimentos específicos e técnicos, mas também a compreensão de um conjunto de normas e requisitos acadêmicos necessários para a progressão e conclusão do percurso letivo. Nesse contexto, os projetos pedagógicos institucionais estabelecem diretrizes estruturais que regulam a aprovação em disciplinas, o cumprimento de carga horária complementar, a realização de estágio supervisionado e o desenvolvimento do Trabalho de Conclusão de Curso (TCC) [Instituto Federal de Educação, Ciência e Tecnologia de Minas Gerais - Campus Ouro Branco 2022].

Para esclarecer dúvidas sobre esses regulamentos, os discentes frequentemente recorrem à navegação nos portais institucionais em busca do Projeto Pedagógico do Curso (PPC) e de ementas, ou entram em contato direto com a coordenação e a secretaria administrativa do campus. No entanto, muitas dessas informações encontram-se dispersas, estruturadas em formatos complexos e permeadas por jargões acadêmicos e jurídicos. Esse cenário dificulta a localização precisa e célere da informação, gerando lapsos de comunicação para o corpo discente e uma sobrecarga de atendimentos repetitivos nos setores de atendimento da instituição [Monteiro 2021].

Do ponto de vista tecnológico, a extração automatizada de informações a partir desses documentos não estruturados, como arquivos no formato PDF, apresenta desafios de engenharia significativos. Modelos de Inteligência Artificial (IA) generativa, como os Modelos de Linguagem de Grande Escala (*Large Language Models* - LLMs), são suscetíveis ao fenômeno de alucinação — a geração probabilística de fatos plausíveis, porém incorretos ou inexistentes —, notadamente quando não possuem o contexto exato e atualizado em sua base fixa de treinamento [Modran 2025]. Portanto, para garantir que um assistente automatizado responda estritamente com base nos regulamentos vigentes, sem inventar regras de desligamento, prazos ou grades curriculares, exige-se uma arquitetura técnica rigorosa que restrinja a geração de texto apenas aos dados provenientes de fontes oficiais e auditadas.

Com o objetivo de mitigar esse problema, a proposta deste trabalho consiste no desenvolvimento de uma aplicação web integrada a um agente de Inteligência Artificial,

projetado para atuar como um assistente especializado nas normas e regulamentos do curso. A intenção central é disponibilizar as informações institucionais por meio de uma interface de chat intuitiva e acessível via internet, visando conferir maior autonomia ao discente e, simultaneamente, reduzir a demanda por atendimentos informativos de rotina nos setores administrativos [Monteiro 2021].

Para viabilizar esse assistente com a precisão factual necessária, o sistema é fundamentado na arquitetura de Geração Aumentada por Recuperação (*Retrieval-Augmented Generation* - RAG). O RAG melhora a confiabilidade dos LLMs ao condicionar a geração da resposta a trechos de documentos específicos recuperados de uma base de conhecimento externa [Modran 2025]. Conceitualmente, o agente opera de forma semelhante ao pessoal de um departamento administrativo que, diante de uma dúvida de um estudante, localiza e consulta o documento oficial correto antes de formular e fornecer a resposta direcionada.

Nesta arquitetura, o fluxo de orquestração e o fornecimento de contexto são padronizados pelo *Model Context Protocol* (MCP) [Sabbag Filho 2025]. O uso do MCP neste projeto visa a consulta a uma base de dados previamente vetorizada com as informações do curso, o que assegura respostas rápidas e seguras sem a necessidade de processar a vetorização dos documentos em tempo real. O pipeline de dados converte os regulamentos oficiais em vetores semânticos (*embeddings*), indexados em um banco de dados vetorial baseado em PostgreSQL com a extensão *pgvector*. Quando o usuário submete uma pergunta, o sistema realiza uma busca híbrida para recuperar estritamente os trechos pertinentes dos regulamentos. O MCP expõe esse contexto restrito de forma segura ao LLM, que sintetiza uma resposta clara e objetiva, informando inclusive a fonte exata da qual a regra foi extraída [Sabbag Filho 2025].

2. Trabalhos Relacionados

Para mitigar problemas de fragmentação da informação e de atendimentos repetitivos em contextos educacionais e administrativos, diversas abordagens têm sido exploradas na literatura técnica e científica. O mapeamento dessas iniciativas permite compreender o panorama de soluções existentes e situar metodologicamente o avanço proposto por este estudo.

No âmbito dos assistentes virtuais voltados ao ecossistema universitário, Monteiro [Monteiro 2021] desenvolveu o *chatbot* "Helena" para o Departamento de Computação da Universidade Federal de Ouro Preto (UFOP). O propósito daquela solução era sanar dúvidas recorrentes de discentes sobre trâmites institucionais integrando as interfaces de comunicação à plataforma IBM Watson Assistant. Do ponto de vista metodológico, o sistema de Monteiro [Monteiro 2021] fundamenta-se em motores de Processamento de Linguagem Natural (PLN) tradicionais, técnica que impõe o mapeamento manual de intenções estruturadas e o desenho de árvores de diálogo estáticas sob regras condicionais rígidas. Em contrapartida, o método proposto no presente trabalho substitui a necessidade de codificação manual de fluxos de decisão pela implementação de uma arquitetura baseada em *Agentic RAG* sobre resoluções oficiais. Essa transição de paradigma elimina o gargalo operacional de manutenção das regras do sistema, delegando ao modelo de linguagem a capacidade de sintetizar respostas de maneira autônoma e dinâmica diretamente dos regulamentos originais.

A evolução das arquiteturas voltadas a cenários educacionais complexos é ilustrada por Modran [Modran 2025], que propõe um tutor inteligente adaptativo estruturado a partir da fusão entre RAG, o *Model Context Protocol* (MCP) e o *Agent Communication Protocol* (ACP). Aquela proposta foca na dimensão pedagógica ao gerenciar o perfil conceitual do estudante para fornecer orientações personalizadas por meio de suporte cognitivo (*scaffolding*). Embora o modelo de comunicação técnica de Modran [Modran 2025] compartilhe semelhanças estruturais com o ecossistema de protocolos deste projeto, os objetivos de aplicação divergem significativamente. Enquanto o referido autor manipula o MCP para injetar heurísticas de aprendizado e adaptar o nível de complexidade do conteúdo ao comportamento do discente, a arquitetura deste trabalho adota o protocolo estritamente como uma camada de governança e segurança de dados. O intuito é mitigar falhas factuais e reter o agente à recuperação fidedigna e sem desvios das diretrizes administrativas.

A contextualização de modelos de linguagem para o esclarecimento de dúvidas acadêmicas e institucionais também foi investigada por Barbosa [Barbosa 2023], que concebeu um assistente conversacional especializado para a Pró-Reitoria de Graduação da UFOP. A arquitetura implementada pelo referido autor baseia-se no uso da biblioteca LlamaIndex em conjunto com a API comercial do ChatGPT (OpenAI), alimentando o modelo generativo por meio de nós textuais extraídos de documentos e manuais estudantis convertidos em arquivos estruturados em formato JSON. Embora o escopo informacional guarde estreita relação com a proposta deste trabalho, as escolhas de engenharia e os pressupostos de infraestrutura diferem de forma drástica. Enquanto o sistema de Barbosa [Barbosa 2023] manifesta total dependência de provedores de nuvem proprietários e processamento externo de APIs pagas, esta pesquisa apoia-se em um ecossistema inteiramente local de código aberto, executando inferências de geração e vetorização em um servidor local via Ollama. Ademais, o modelo de busca de Barbosa [Barbosa 2023] limita-se à recuperação linear de documentos gerenciados em memória volátil, ao passo que a solução aqui descrita introduz um pipeline híbrido persistente baseado em busca vetorial e léxica acoplado à orquestração agêntica via protocolo MCP.

A validação do MCP como plano de controle (*control plane*) e abstração arquitetural em ambientes corporativos e distribuídos também é documentada por Oliveira [Oliveira 2025]. O autor projetou uma arquitetura multiagente voltada ao provisionamento e à gerência automatizada de infraestruturas OpenStack, demonstrando a viabilidade de expor capacidades computacionais (*tools e resources*) a modelos de linguagem sob restrições e contratos formais de interface. Apesar de o trabalho de Oliveira [Oliveira 2025] confirmar o potencial do protocolo em delimitar o campo de atuação de agentes de inteligência artificial, o escopo operacional desta pesquisa impõe restrições adicionais de design. Ao passo que a infraestrutura multiagente do citado autor realiza operações ativas com efeitos colaterais no ambiente físico e virtual, como a criação e exclusão de instâncias de servidores, o sistema aqui detalhado atua sob um regime estrito de leitura. Essa barreira de segurança passiva visa assegurar a integridade do domínio informacional prestado ao corpo discente.

A consolidação teórica e a padronização desses modelos de comunicação organizacional são discutidas por Sabbag Filho [Sabbag Filho 2025], que define o MCP como um mecanismo de contratos explícitos destinado a reduzir ambiguidades e vulnerabili-

dades na exposição de dados internos a agentes de IA. As diretrizes estabelecidas por Sabbag Filho [Sabbag Filho 2025] servem de alicerce para a governança deste projeto, justificando o emprego do protocolo para orquestrar o tráfego de dados entre a camada de busca semântica — hospedada em um banco de dados vetorial PostgreSQL com a extensão *pgvector* — e o modelo de linguagem. O protocolo atua, portanto, como o elo de previsibilidade e isolamento da base de conhecimento acadêmica, viabilizando o fornecimento seguro de contexto ao usuário.

3. Fundamentação Teórica

Esta seção apresenta o embasamento teórico indispensável para a compreensão do ecossistema tecnológico empregado neste trabalho. São detalhados os conceitos que norteiam o uso de modelos de linguagem de grande escala, os mecanismos de recuperação de dados e a governança de interfaces que asseguram a confiabilidade da aplicação desenvolvida.

3.1. Modelos de Linguagem e o Fenômeno da Alucinação

Os Modelos de Linguagem de Grande Escala (*Large Language Models* - LLMs) representam uma evolução significativa no campo do Processamento de Linguagem Natural (PLN). Fundamentados em arquiteturas de redes neurais profundas, como o *Transformer*, esses modelos são treinados sobre volumes massivos de dados textuais para prever a probabilidade da próxima palavra ou fragmento de texto (*token*) em um determinado contexto. Embora demonstrem uma capacidade notável de síntese e geração de linguagem natural fluida, os LLMs operam de maneira probabilística, carecendo de uma compreensão factual ou estrita do mundo real.

Essa natureza estatística expõe os modelos ao fenômeno técnico conhecido como alucinação. A alucinação ocorre quando o LLM gera saídas que são gramaticalmente corretas e semanticamente plausíveis, porém factualmente incorretas ou totalmente desconectadas da realidade histórica contida em seus pesos neurais. Em sistemas de informação projetados para domínios institucionais, jurídicos ou administrativos — onde a precisão da informação é mandatória —, a ocorrência de alucinações invalida o uso de abordagens baseadas exclusivamente em modelos generativos puros, demandando mecanismos externos de controle e contenção de contexto.

3.2. Geração Aumentada por Recuperação (RAG)

Para mitigar o problema das alucinações factuais sem a necessidade de reprocessar ou realizar o ajuste fino (*fine-tuning*) oneroso dos parâmetros de um LLM, consolidou-se na literatura a arquitetura de Geração Aumentada por Recuperação (*Retrieval-Augmented Generation* - RAG). O princípio fundamental do RAG consiste em desacoplar o conhecimento paramétrico do modelo (adquirido na fase de treinamento) do conhecimento não-paramétrico (armazenado em uma base de dados externa dinâmica) [Lewis et al. 2020]

O fluxo operacional do RAG divide-se essencialmente em três etapas consecutivas: recuperação, aprofundamento e geração. Diante de uma requisição enviada pelo usuário, o sistema inicialmente consulta o repositório externo de documentos para extrair os trechos informativos semanticamente relevantes. Em seguida, esses fragmentos textuais recuperados são injetados no *prompt* original, expandindo o contexto fornecido ao

modelo. Por fim, o LLM atua estritamente como um sintetizador de linguagem, processando o bloco de dados delimitado e gerando uma resposta ancorada exclusivamente nas evidências documentais anexadas.

3.3. Bancos de Dados Vetoriais e Busca Semântica Híbrida

A viabilização do RAG baseia-se na indexação e busca eficiente de documentos não estruturados através de vetores matemáticos de alta dimensionalidade, denominados *embeddings*. O processo de geração de *embeddings* envolve a submissão dos fragmentos textuais a um modelo de representação numérica capaz de mapear o significado semântico das palavras em um espaço vetorial contínuo. Desse modo, trechos textuais que compartilham ideias semelhantes são posicionados geometricamente próximos uns dos outros, independentemente das palavras exatas ou da sintaxe utilizada. Para este trabalho, adota-se o modelo *embeddings* local *bge-m3*, caracterizado por gerar vetores densos estruturados em 1024 dimensões [Lewis et al. 2020]. A recuperação dessas estruturas numéricas ocorre por meio de bancos de dados vetoriais relacionais, utilizando a extensão *pgvector* integrada ao ecossistema PostgreSQL com índices do tipo *Hierarchical Navigable Small World* (HNSW). O índice HNSW organiza os vetores em um grafo multicamadas, permitindo a execução de buscas aproximadas de vizinhos mais próximos (*Approximate Nearest Neighbors* - ANN) com complexidade logarítmica. Essa abordagem foi preferida em relação ao modelo *Inverted File Flat* (IVFFlat) por não requerer uma fase de treinamento prévio de partições e por manter alta revocação (*recall*) e reduzida latência, mesmo sob atualizações e inserções frequentes de novos documentos na base de conhecimento acadêmica.

$$Score_{RRF}(d \in D) = \alpha \cdot \frac{1}{k + rank_{semântico}(d)} + (1 - \alpha) \cdot \frac{1}{k + rank_{lexical}(d)} \quad (1)$$

Em que d denota um fragmento contido no espaço de resultados D , $rank(d)$ expressa a posição ordinal de classificação do fragmento dentro de seu respectivo método de busca, e k constitui a constante de suavização fixada em 60. O fator α permite calibrar a relevância de acordo com o pipeline de processamento da arquitetura: no RAG Clássico linear, adota-se $\alpha = 0,5$ para estabelecer um equilíbrio idêntico entre semântica e correspondência de termos; na busca do Servidor MCP, utiliza-se $\alpha = 0,4$, conferindo um peso superior à busca lexical para garantir maior precisão no resgate de siglas ou termos específicos do regulamento acadêmico sob a governança das ferramentas do agente.

3.4. O Model Context Protocol (MCP)

Embora o pipeline de indexação vetorial resolva a extração da informação, a integração segura e padronizada entre os modelos de inteligência artificial e as fontes de dados corporativas ou institucionais representa um desafio crítico de engenharia de software. Historicamente, essa comunicação era realizada por meio de implementações *ad hoc*, gerando forte acoplamento e vulnerabilidades na exposição de barramentos de dados. Para padronizar essa camada, estabeleceu-se o *Model Context Protocol* (MCP) [Sabbag Filho 2025]. Conforme analisado por Sabbag Filho [Sabbag Filho 2025], o MCP opera como uma abstração arquitetural que define contratos formais de interface baseados no modelo cliente-servidor. O protocolo visa unificar a maneira como os agentes

inteligentes consomem dados organizacionais externos por meio de três primitivas principais de comunicação:

- **Resources:** Fontes de dados textuais ou dinâmicas controladas, como as ementas e as resoluções de curso, expostas ao modelo em formato de leitura segura e contextualizada;
- **Tools:** Ações ou ferramentas computacionais executáveis expostas explicitamente ao modelo, permitindo que o agente invoque buscas parametrizadas ativas sob regras estritas de contrato;
- **Prompts:** Modelos de instruções e orientações pré-configurados que auxiliam na padronização e na ancoragem da comunicação interna da aplicação.

O emprego do MCP estabelece uma fronteira rígida de governança ao redor do LLM. Em vez de conceder ao modelo acesso direto ou irrestrito à infraestrutura de dados do servidor local, o protocolo assegura que toda e qualquer interação ocorra sob esquemas declarativos rígidos e contratos de interface bem definidos. No contexto de sistemas de informação acadêmicos, essa abordagem impede desvios de escopo gerados por interações coloquiais de usuários e garante a previsibilidade necessária para consultas administrativas e jurídicas de alta relevância.

Colocar trabalhos relacionados semelhantes ao nosso, ou seja, tentativas alternativas na literatura de resolver o mesmo problema**.

O objetivo é "comparar e contrastar" - como a abordagem deles difere em termos de pressupostos ou método? Se o método for aplicável ao nosso problema, espero uma comparação na seção experimental. Caso contrário, deve haver uma declaração clara de por que um determinado método não é aplicável. Observação: apenas descrever o que outro artigo está fazendo não é suficiente. Precisamos comparar e contrastar.

Exemplo abaixo:

Background: Antecedentes acadêmicos do nosso trabalho, ou seja, todos os conceitos e trabalhos anteriores necessários para a compreensão do nosso método. Geralmente inclui uma subseção, Definição do Problema, que apresenta formalmente a definição do problema e a notação (Formalismo) do nosso método. Destaca quaisquer pressupostos específicos que sejam incomuns. **Ou seja, tudo que deve ser escrito acima. Ele só tá desenhando como deve fazer.

4. Metodologia

A fazer futuramente.

A metodologia adotada neste trabalho fundamenta-se no desenvolvimento e na engenharia de um ecossistema de software estruturado sob a filosofia de monorepo, utilizando *npm workspaces*. O sistema foi concebido para viabilizar e avaliar duas abordagens concorrentes de recuperação de informações acadêmicas em um ambiente local de hardware modesto, permitindo analisar de forma comparativa a viabilidade de pipelines lineares em relação a arquiteturas agênticas autônomas.

4.1. Abordagens Arquiteturais Comparativas

O ecossistema implementa dois fluxos computacionais paralelos para o processamento das consultas formuladas pelos discentes. A primeira abordagem consiste em um pipeline

RAG Clássico linear e determinístico. O fluxo inicia-se com o recebimento da consulta do aluno pelo servidor Express em TypeScript, disparando uma etapa de reescrita de consulta (*Query Rewriting*) apoiada por um LLM leve. Esse processo expande siglas acadêmicas padronizadas do campus e categoriza a intenção em rótulos restritos. Na sequência, a consulta expandida é convertida em um vetor denso pelo modelo *bge-m3* e submetida à busca híbrida com RRF no PostgreSQL. Os fragmentos textuais resultantes são injetados no *prompt* de sistema como contexto estrito para a síntese final do modelo generativo, transmitida ao discente via *Server-Sent Events* (SSE) em modo de fluxo contínuo (*streaming*).

A segunda abordagem estrutura-se sob o paradigma de *Agentic RAG* mediado pelo protocolo MCP. Nessa configuração, o servidor Express inicializa um cliente MCP que estabelece um canal de comunicação reativo baseado em subprocesso *stdio* com um servidor MCP autônomo. O servidor MCP publica dinamicamente a ferramenta *search_ifmg_knowledge*, mapeando-a para a especificação de chamada de função (*Function Calling*) esperada pelo LLM. Ao receber a pergunta do discente, o modelo opera como um agente autônomo, avaliando o *prompt* para decidir deliberadamente se há necessidade de acionar a ferramenta de busca ou se a interação pode ser respondida diretamente de forma interna. Caso a busca seja invocada, o servidor MCP executa o pipeline híbrido no PostgreSQL, aplicando uma nota de corte estrita baseada em um limite mínimo de pontuação RRF para descartar ruídos textuais de baixo ranking antes de retornar as evidências para a síntese final do agente.

4.2. Pipeline de Ingestão e Sanitização de Documentos

Para garantir a qualidade dos dados contidos na base de conhecimento e evitar a poluição do espaço vetorial, estruturou-se um fluxo de ingestão de documentos não estruturados dividido em fases de extração, limpeza e fragmentação adaptativa. Arquivos nos formatos PDF ou de imagem passam por processamento assistido por inteligência de visão espacial e reconhecimento óptico de caracteres (*OCR*) local via *tesseract.js*, convertendo tabelas físicas em notações textuais estruturadas em Markdown. O texto bruto resultante é submetido a um serviço de sanitização dedicado composto por oito etapas sequenciais:

- **Limpeza de OCR:** Remoção de caracteres de controle inválidos, linhas em branco e resíduos de codificação Unicode;
- **Reconstituição de Hifenização:** Junção gramatical automatizada de termos que foram truncados com hífen devido à transição física de linhas nos arquivos originais;
- **Remoção de Elementos Institucionais:** Eliminação de cabeçalhos burocráticos repetitivos, e-mails, endereços e telefones por meio de expressões regulares;
- **Poda de Anexos:** Truncamento automático do fluxo textual ao detectar marcadores de início de apêndices, isolando formulários vazios que causariam ruído semântico;
- **Conversão de Tabelas:** Normalização de dados tabulares em texto corrido contíguo para assegurar que códigos e nomes de matérias compartilhem a mesma vizinhança vetorial;
- **Limpeza Estrutural:** Exclusão de numerações isoladas de páginas e caracteres de controle decorativos;

- **Preparação Jurídica:** Interconexão de linhas isoladas de artigos e injeção de quebras duplas de parágrafo antes de marcadores de seções legais, otimizando o fracionamento lógico;
- **Normalização Final:** Consolidação de múltiplos espaços vazios e exclusão de linhas espúrias ruidosas.

Após a sanitização, o texto é processado por um algoritmo de *chunking* semântico adaptativo, aplicando divisões específicas com base no formato documental detectado: estruturas jurídicas são preservadas integralmente mantendo artigos e incisos agregados; dados tabulares replicam as linhas de cabeçalho em cada sub-bloco; e textos gerais sofrem fracionamento por parágrafo com sobreposições (*overlap*) de segurança. Por fim, cada fragmento recebe a injeção automática de um prefixo de contexto global contendo os metadados do documento de origem antes de sua conversão vetorial e armazenamento persistente.

4.3. Mecanismo de Controle de Concorrência e VRAM

Considerando as restrições físicas de computação em hardware de homelab universitário local, a concorrência não controlada de múltiplas requisições simultâneas de inferência pesada causaria falhas por saturação e estouro de memória física (*Out of Memory* - OOM) na GPU dedicada, paralisando o *daemon* do Ollama. Para solucionar essa vulnerabilidade de infraestrutura, implementou-se um semáforo de controle de concorrência ativa gerenciado por filas de tarefas com Redis e BullMQ.

O mecanismo encapsula o fluxo de geração por meio de uma lógica de semáforo de aquisição e liberação (*acquire/release*). O sistema limita o número de processamentos paralelos ao máximo parametrizado de acordo com a capacidade física do chip gráfico do servidor. Caso o limite seja atingido, as requisições excedentes de conexões SSE entram em um estado de espera controlada em uma fila de memória por um intervalo limite de tempo, garantindo a estabilidade operacional e a alta disponibilidade do assistente sem comprometer a integridade física do servidor institucional.

5. Experimentos e Resultados

A fazer futuramente. Caso necessário, dividir as seções

6. Conclusão

A fazer futuramente .

Referências

Barbosa, L. S. (2023). Um chatbot especializado para o contexto da universidade federal de ouro preto. Monografia (bacharelado em ciência da computação), Universidade Federal de Ouro Preto (UFOP), Ouro Preto, MG.

Instituto Federal de Educação, Ciência e Tecnologia de Minas Gerais - Campus Ouro Branco (2022). Projeto pedagógico do curso de bacharelado em sistemas de informação. https://www.ifmg.edu.br/ourobranco/nossos-cursos/copy_of_PPCBSI_2022.pdf. Acessado em: 22 abr. 2026.

- Lewis, P., Perez, E., Piktus, A., Petroni, F., Lewis, V., Das, P., Riedel, S., and Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474.
- Modran, H. A. (2025). Leveraging rag with acp & mcp for adaptive intelligent tutoring. *Applied Sciences*, 15(21):11443.
- Monteiro, G. S. (2021). Helena: Um chatbot para auxílio dos discentes do decom em trâmites universitários. Monografia (bacharelado em ciência da computação), Universidade Federal de Ouro Preto, Ouro Preto, MG.
- Oliveira, I. K. d. S. (2025). Uma arquitetura multiagente para orquestração automática em infraestruturas openstack. Trabalho de Conclusão de Curso (Bacharelado em Engenharia de Computação) – Instituto Federal de Educação, Ciência e Tecnologia da Paraíba (IFPB).
- Sabbag Filho, N. (2025). Model context protocol (mcp): Conectando contexto, agentes e arquitetura de software moderna. *Leaders.Tec.Br*, 2(37):1–12.